B.Sc. Information Technology — Semester VI

Internet of Things (IoT)

Course Code: 25UBIT603

UNIT 2

Sensors, Actuators & Microcontrollers

Topics Covered:

① Types of Sensors — Temperature, Motion, Proximity, Gas
② Actuators — Motors, Relays, Valves
③ Introduction to Arduino and Raspberry Pi
④ GPIO Programming
⑤ Sensor-Controller Interfacing

IoT Exam: 27th March | Credits: 4 | Max Marks: 150

1. Types of Sensors

1.1 What is a Sensor?

A sensor is an input device that detects or measures a physical quantity (like temperature, light, motion) and converts it into an electrical signal that a microcontroller can process.

> 📌 *Analogy: Sensors are the 'sense organs' of IoT — just like eyes, ears, nose, skin gather information for humans*

1.2 Sensor Classification

Classification Basis	Types	Examples
Output Signal Type	Analog (continuous voltage) vs Digital (0 or 1)	LM35 = analog | DHT11 = digital
Energy Source	Active (needs external power) vs Passive (self-powered)	Thermistor = passive | Ultrasonic = active
Physical Quantity	Temperature, Light, Pressure, Motion, Gas, Humidity	See categories below
Contact with Measurand	Contact vs Non-contact	Thermocouple = contact | IR sensor = non-contact

1.3 Temperature Sensors

Purpose: Measure thermal energy of an object/environment and convert to electrical signal.

Sensor	Output	Temp Range	Accuracy	Interface	Best For
LM35	Analog	-55 to +150°C	±0.5°C	ADC pin	Simple analog circuits
DHT11	Digital	0 to +50°C	±2°C	1-Wire	Basic projects, cheap
DHT22	Digital	-40 to +80°C	±0.5°C	1-Wire	Better accuracy needed
DS18B20	Digital	-55 to +125°C	±0.5°C	1-Wire (multi)	Waterproof, multiple sensors
Thermocouple K	Analog (µV)	-200 to +1372°C	±2°C	SPI (MAX6675)	Extreme temperatures

1.4 Motion Sensors

Purpose: Detect movement of objects or people in the sensor's detection zone. Critical for security systems, automatic lighting, and gesture recognition.

Sensor	Type	Range	Output	Use Case
PIR (HC-SR501)	Passive Infrared	3-7m, 120° angle	Digital (HIGH=motion)	Security alarm, auto lighting

Sensor	Type	Range	Output	Use Case
HC-SR04 Ultrasonic	Sound-based	2-400 cm	Echo pulse duration	Distance measurement, obstacle avoidance
MPU-6050	Accelerometer + Gyroscope	±2g to ±16g	I2C digital	Motion tracking, drone stabilization
ADXL345	Accelerometer only	±2g to ±16g	I2C/SPI	Vibration detection, tilt sensing

📌 *HC-SR04 Formula: Distance (cm) = pulseIn(ECHO, HIGH) × 0.0343 / 2*

1.5 Proximity Sensors

Purpose: Detect presence or absence of an object within a certain distance WITHOUT physical contact. Essential for industrial automation, touchless interfaces, and obstacle avoidance.

Type	Working Principle	Range	Application
Infrared (IR)	Emits IR light, detects reflection	2-30 cm	Object detection, line following robot
Ultrasonic	Sends sound waves, measures echo time	2-400 cm	Parking sensors, obstacle avoidance
Capacitive	Detects change in capacitance	1-10 mm	Touch screens, liquid level detection
Inductive	Detects metal objects via magnetic field	1-50 mm	Industrial metal detection, CNC machines

1.6 Gas Sensors (MQ Series)

Purpose: Detect presence and concentration of specific gases. Critical for safety monitoring (leak detection), air quality assessment, and industrial process control.

Sensor	Target Gas	Detection Range	Application
MQ-2	LPG, Propane, Methane, Smoke, H2	300-10000 ppm	LPG leak alarm, smoke detector
MQ-3	Alcohol (Ethanol)	0.05-10 mg/L	Breathalyzer, drunk-driving prevention
MQ-4	Methane (Natural Gas)	300-10000 ppm	Natural gas leak detection
MQ-5	LPG, Natural Gas	300-10000 ppm	Fuel gas leak detector
MQ-7	Carbon Monoxide (CO)	20-2000 ppm	CO poisoning alarm, vehicle exhaust
MQ-135	Air Quality (NH3, CO2, Benzene)	10-300 ppm	Indoor air quality monitor

How MQ Gas Sensors Work:

- Sensing element = ceramic tube coated with metal oxide (SnO_2 — tin dioxide)
- Internal heater heats element to 200-300°C for optimal gas detection
- When target gas contacts surface → reduces resistance of sensing element
- Higher gas concentration → lower resistance → higher analog voltage output
- Requires 48 hour burn-in (first use) + 2-3 minutes warm-up before each use

---

2. Actuators — Motors, Relays, Valves

2.1 What is an Actuator?

An actuator is an output device that receives an electrical signal from a microcontroller and converts it into a physical action. Actuators are the 'hands' of IoT — they make things happen in the physical world.

> 📌 *Sensors = Input devices (sense → electrical signal) | Actuators = Output devices (electrical signal → action)*

2.2 Motors

Feature	DC Motor	Servo Motor	Stepper Motor
Control Type	Speed + Direction (PWM)	Angular Position (PWM)	Step-by-step position (digital)
Precision	Low — no position feedback	High — ±1-2°	Very High — fraction of degree
Feedback	Open loop (no feedback)	Closed loop (built-in pot)	Open loop (counts steps)
Speed	High RPM possible	Limited to ~60-120 RPM	Low to medium RPM
Driver IC	L298N / L293D	Direct PWM from MCU	ULN2003 / A4988
Best For	Wheels, fans, pumps	Robot joints, RC control	3D printers, CNC, precision positioning

2.3 Relays

A relay is an electrically controlled switch that allows a low-power microcontroller signal to control high-power circuits (like 220V AC appliances).

How a Relay Works:

- Microcontroller sends LOW/HIGH signal to relay's input pin
- This activates a transistor which allows current through relay's electromagnet coil
- Electromagnetic field attracts ferromagnetic armature — switches mechanical contacts
- When signal removed — spring returns armature, flyback diode protects circuit

Contact Type	Description	Use Case
NO (Normally Open)	Contact OPEN when relay NOT energized — CLOSES when activated	Circuit OFF by default — turns ON when relay triggers (most common)
NC (Normally Closed)	Contact CLOSED when relay NOT energized — OPENS when activated	Circuit ON by default — turns OFF when relay triggers
COM (Common)	Common terminal shared between NO and NC — connected to load power	Always connected to load's power supply

> 📌 *Most relay modules are Active LOW — GPIO LOW = relay ON, GPIO HIGH = relay OFF*

2.4 Solenoid Valves

A solenoid valve is an electromechanical actuator that controls the flow of liquids or gases. An electromagnetic coil moves a plunger to open or close the valve.

Valve Type	Description	Application
Normally Closed (NC)	CLOSED when de-energized, OPENS when current applied — most common for fail-safe	Smart irrigation — stops water if power fails
Normally Open (NO)	OPEN when de-energized, CLOSES when current applied	Used where flow should continue if power fails
2-Way Valve	One inlet, one outlet — simply opens or closes flow	Basic water/gas flow control
3-Way Valve	Three ports — can divert flow between two paths	Mixing or diverting applications

📌 *Solenoid valves require 12V or 24V — must be controlled via relay module from 5V Arduino GPIO*

3. Arduino and Raspberry Pi

3.1 Arduino Uno — Hardware Specifications

Component	Specification	Importance
Microcontroller	ATmega328P (8-bit AVR)	The brain — runs your code
Clock Speed	16 MHz	~16 million instructions per second
Flash Memory	32 KB	Stores compiled sketch code
SRAM	2 KB	Runtime memory for variables — very limited
EEPROM	1 KB	Non-volatile storage — survives power off
Operating Voltage	5V	Logic level — input pins operate at 5V
Digital I/O Pins	14 pins (D0-D13)	Each can source/sink max 40mA
PWM Pins	6 pins (~3,5,6,9,10,11)	8-bit PWM — use analogWrite()
Analog Input Pins	6 pins (A0-A5)	10-bit ADC — reads 0-5V as 0-1023
I2C Pins	A4 (SDA), A5 (SCL)	For I2C communication
UART Pins	D0 (RX), D1 (TX)	Serial communication

3.2 Arduino Programming Fundamentals

Two Mandatory Functions in Every Sketch:

Function	When it Runs	Used For
setup()	Runs ONCE when Arduino powers on or is reset	Initializing pins, starting Serial, configuring sensors
loop()	Runs CONTINUOUSLY in infinite loop after setup()	Reading sensors, controlling actuators, main program logic

Core Arduino Functions:

Function	Syntax	Description
pinMode()	pinMode(pin, mode)	Sets pin as INPUT, OUTPUT, or INPUT_PULLUP — must call in setup()
digitalWrite()	digitalWrite(pin, value)	Sets digital output pin HIGH (5V) or LOW (0V)
digitalRead()	int val = digitalRead(pin)	Reads digital input — returns HIGH (1) or LOW (0)
analogRead()	int val = analogRead(pin)	Reads 0-5V on A0-A5 → returns 0-1023 (10-bit ADC)
analogWrite()	analogWrite(pin, value)	Outputs PWM (0-255) on PWM pins — 0=0V, 255=5V
delay()	delay(ms)	Pauses program for milliseconds — blocking

Function	Syntax	Description
millis()	unsigned long t = millis()	Returns ms since Arduino started — non-blocking timer
Serial.begin()	Serial.begin(9600)	Starts serial communication at baud rate
Serial.println()	Serial.println(data)	Prints data to Serial Monitor with newline

3.3 Raspberry Pi Models Overview

Model	Processor	RAM	Connectivity	Key Features
RPi Zero 2 W	Quad-core A53 @ 1 GHz	512 MB	Wi-Fi, BT 4.2	Ultra-compact, low power — tiny IoT projects
RPi 3 Model B+	Quad-core A53 @ 1.4 GHz	1 GB	Wi-Fi ac, BT 4.2, GbE	Most educational RPi — widely used
RPi 4 Model B	Quad-core A72 @ 1.5 GHz	1/2/4/8 GB	Wi-Fi ac, BT 5.0, GbE	Desktop-class performance, USB 3.0, 4K HDMI
RPi 5	Quad-core A76 @ 2.4 GHz	4/8 GB	Wi-Fi ac, BT 5.0, GbE	2-3x faster than RPi 4, PCIe for NVMe
RPi Pico (W)	Dual-core M0+ @ 133 MHz	264 KB SRAM	Pico W: Wi-Fi, BT	Microcontroller — like Arduino, MicroPython

3.4 Raspberry Pi vs Arduino — Comparison

Feature	Raspberry Pi 4	Arduino Uno
Category	Single-Board Computer (SBC)	Microcontroller Development Board
Processor	Quad-core ARM Cortex-A72, 64-bit, 1.5 GHz	8-bit AVR ATmega328P, 16 MHz
RAM	1 GB - 8 GB LPDDR4	2 KB SRAM
Operating System	Linux (Raspberry Pi OS), Ubuntu	No OS — runs one program directly
Programming	Python, C/C++, Java, Node.js — any language	C/C++ (Arduino IDE)
Multitasking	Yes — full OS multitasking	No — single program only
Internet	Built-in Wi-Fi + Gigabit Ethernet	Requires add-on shield
GPIO Voltage	3.3V — NOT 5V tolerant!	5V (also 3.3V on some models)
Startup Time	30-60 seconds (Linux boot)	Instant (<1 second)
Real-Time Control	Not ideal — OS scheduling delays	Excellent — bare metal, deterministic timing
Power Consumption	3-7 Watts	0.05-0.25 Watts

Feature	Raspberry Pi 4	Arduino Uno
Best For	Gateway, edge AI, computer vision, web server	Sensor reading, motor control, real-time IoT nodes

4. GPIO Programming

4.1 GPIO as Digital OUTPUT

Concept	Explanation
Source Current (HIGH)	When OUTPUT pin is HIGH (5V), current flows FROM pin INTO circuit. Max 40mA per pin on Arduino
Sink Current (LOW)	When OUTPUT pin is LOW (0V), current flows INTO pin FROM circuit. Also max 40mA
Total GPIO Current	Total current across ALL GPIO pins should not exceed 200mA — use drivers for high-current loads
LED Connection	LED on OUTPUT pin: connect 220Ω-1kΩ resistor in series to limit current
Relay/Motor Control	Always use driver ICs (L298N, relay module, ULN2003) for high-current loads — NEVER connect directly to GPIO
PWM Output	Rapidly switches HIGH/LOW at varying duty cycles to simulate analog voltage. 8-bit = 256 levels

4.2 GPIO as Digital INPUT

Input Mode	Description
INPUT (floating)	Reads external voltage — when nothing connected pin 'floats' and randomly reads HIGH or LOW — AVOID for switches
INPUT_PULLUP	Internal 10-20kΩ resistor connects pin to VCC (5V). Pin reads HIGH when switch open, LOW when pressed. RECOMMENDED for switches
Debouncing	Physical switches 'bounce' for 5-20ms when pressed. Implement software debounce using millis() or delay(50ms) to avoid false readings

4.3 PWM — Pulse Width Modulation

PWM rapidly switches a pin HIGH/LOW at varying duty cycles to simulate an analog voltage output.

PWM Concept	Explanation
Duty Cycle (%)	Percentage of time signal is HIGH. Duty Cycle = (ON time / Period) × 100%
analogWrite(pin, 0)	0% duty cycle — signal always LOW — effective output = 0V
analogWrite(pin, 128)	~50% duty cycle — half HIGH, half LOW — effective output ≈ 2.5V
analogWrite(pin, 255)	100% duty cycle — signal always HIGH — effective output = 5V
LED Brightness	PWM duty cycle directly controls perceived LED brightness
Motor Speed	Average voltage ∝ duty cycle → controls motor speed smoothly

4.4 GPIO on Raspberry Pi

Pin Type	Voltage	Max Current	Description
GPIO (Digital I/O)	3.3V logic	8-16 mA per pin	Programmable digital I/O — NOT 5V tolerant! 5V input will damage the Pi
5V Power Pins	5V	Limited by supply	Pins 2 and 4 — provide 5V power output (not GPIO)
3.3V Power Pins	3.3V	~50 mA total	Pins 1 and 17 — provide 3.3V power output
I2C	3.3V	—	GPIO 2 (SDA), GPIO 3 (SCL) — I2C Bus 1
SPI	3.3V	—	GPIO 10 (MOSI), GPIO 9 (MISO), GPIO 11 (SCLK)
UART	3.3V	—	GPIO 14 (TXD), GPIO 15 (RXD) — serial communication
PWM	3.3V	—	GPIO 12, 13, 18, 19 have hardware PWM capability

5. Sensor-Controller Interfacing

5.1 Communication Protocols for Sensor Interfacing

Protocol	Wires	Speed	Devices	Best For
GPIO Digital	1 data + GND	Instantaneous	1 per pin	Buttons, LEDs, relays, digital sensors
ADC (Analog)	1 data + GND	Instantaneous	1 per ADC pin	LM35, potentiometers, analog sensors
UART / Serial	2 (TX, RX) + GND	9600-115200 bps	1 per UART	GPS, Bluetooth HC-05, serial displays
I2C / TWI	2 (SDA, SCL) + GND	100K/400K/1M bps	Up to 127 devices	MPU6050, OLED, BMP280 — multiple sensors on 2 wires
SPI	4 (MOSI, MISO, SCK, CS) + GND	Up to 10 MHz+	Multiple (1 CS per device)	SD cards, TFT displays, MAX6675 — fast data
1-Wire	1 data + GND	15.4 Kbps	Multiple (64-bit IDs)	DS18B20 temperature sensor
PWM	1 PWM + GND	490-50kHz	1 per PWM pin	Servo motors, LED brightness, motor speed

5.2 I2C Protocol — Key Details

I2C Feature	Details
SDA (Serial Data)	Bidirectional data line — carries data between master and slave
SCL (Serial Clock)	Unidirectional clock line — always driven by master to synchronize data
Pull-up Resistors	Both SDA and SCL REQUIRE pull-up resistors (typically 4.7kΩ) to VCC — without them bus won't work
Address Frame	First 7 bits = device address, last bit = R/W (0 = write, 1 = read)
Speed Modes	Standard: 100 kbps | Fast: 400 kbps | Fast-Plus: 1 Mbps
Arduino I2C Pins	Uno: A4 (SDA), A5 (SCL) | Mega: 20 (SDA), 21 (SCL) | Use Wire.h library
Raspberry Pi I2C	GPIO 2 (SDA), GPIO 3 (SCL) — Bus 1 | Use smbus2 library in Python

5.3 Common I2C Sensor Addresses

Sensor / Module	I2C Address	Function
MPU-6050	0x68 (default) or 0x69	6-axis IMU — Accelerometer + Gyroscope
BMP280 / BME280	0x76 or 0x77	Temperature + Pressure (+ Humidity for BME280)
SSD1306 OLED	0x3C or 0x3D	128×64 OLED display
DS3231 RTC	0x68	Real-Time Clock

Sensor / Module	I2C Address	Function
ADS1115	0x48-0x4B	16-bit 4-channel ADC expander for analog sensors

5.4 Practical Sensor Interfacing Examples

Sensor	Connections	Key Formula / Note
DHT11 Temp+Humidity	VCC→5V | DATA→D2 (+10kΩ pullup) | GND→GND	Use Adafruit DHT library | Read once per second max
HC-SR04 Ultrasonic	VCC→5V | TRIG→D9 | ECHO→D10 | GND→GND	Distance(cm) = pulseIn(ECHO,HIGH) × 0.0343 / 2
PIR Motion Sensor	VCC→5V | OUT→D7 | GND→GND	OUT=HIGH when motion detected | Set sensitivity with potentiometer
MQ-2 Gas Sensor	VCC→5V | AOUT→A0 | DOUT→D8 | GND→GND	Preheat 48hrs (first use) + 3min warm-up | Higher voltage = more gas
Relay Module	VCC→5V | IN→D7 | GND→GND	Active LOW: digitalWrite(7, LOW) = relay ON | Use for AC appliances

Unit 2 — Quick Revision Summary

Topic	Key Points
Sensor	Input device | Detects physical property → converts to electrical signal
Actuator	Output device | Receives electrical signal → converts to physical action
LM35	Analog temp | 10mV/°C | -55 to +150°C | ADC pin
DHT11	Digital temp+humidity | 0-50°C ±2°C | 20-80% RH | 1-Wire protocol
DHT22	Better DHT11 | -40 to +80°C ±0.5°C | Every 2 seconds
DS18B20	Waterproof | 1-Wire | Multi-drop | -55 to +125°C
PIR Sensor	Detects IR from warm bodies | Digital output | HC-SR501 | 3-7m, 120°
HC-SR04	Ultrasonic distance | TRIG=10µs pulse | Distance = echo × 0.0343/2 | 2-400cm
MQ Gas Sensors	Metal oxide semiconductor | 48hr burn-in + 3min warm-up | MQ-2=LPG/smoke, MQ-7=CO, MQ-135=air quality
DC Motor	Speed via PWM (0-255) | Direction via L298N H-bridge | Cannot connect directly to GPIO
Servo Motor	Position via PWM | 1ms=0° | 1.5ms=90° | 2ms=180° | 50Hz | Servo.h
Stepper Motor	Precise steps | 28BYJ-48 = 2048 steps/rev | Driver = ULN2003
Relay	NO = off by default | NC = on by default | COM = common | Active LOW
Solenoid Valve	Electromagnet moves plunger | 12V/24V | Controlled via relay
Arduino Uno	ATmega328P | 16MHz | 2KB RAM | 32KB Flash | 14 digital | 6 ADC | 6 PWM | 5V
setup() vs loop()	setup() runs once | loop() runs forever | Both mandatory in every sketch
analogRead()	Reads 0-5V on A0-A5 → returns 0-1023 (10-bit ADC)
PWM	analogWrite(pin, 0-255) | 0=0V | 128≈2.5V | 255=5V | Controls LED/motor
RPi GPIO	3.3V logic — NOT 5V tolerant! | BCM numbering | I2C on GPIO 2(SDA) & 3(SCL)
I2C Protocol	SDA + SCL | 2 wires | 127 devices | Pull-up resistors 4.7kΩ required | Wire.h
SPI Protocol	MOSI, MISO, SCK, CS | 4 wires | Fast | 1 CS pin per device | Full-duplex
1-Wire Protocol	Single data wire | Unique 64-bit device IDs | DS18B20 uses this

Important Exam Questions

- What is a Sensor? Explain types of sensors with classification table (5 marks)
- Compare DHT11, DHT22 and DS18B20 temperature sensors (5 marks)
- Explain MQ gas sensor series — how it works and different variants (5 marks)
- What is an Actuator? Explain DC motor, Servo motor and Stepper motor comparison (7 marks)
- Explain Relay — how it works, NO/NC/COM contacts with applications (5 marks)

- Compare Arduino Uno and Raspberry Pi in detail (7 marks)
- Explain GPIO programming — OUTPUT, INPUT, PWM with examples (5 marks)
- Explain I2C protocol — SDA, SCL, pull-up resistors, address frame (5 marks)
- Compare all sensor interfacing protocols — GPIO, UART, I2C, SPI, 1-Wire (7 marks)

✅ **All the best for IoT Exam on 27th March! Unit 2 — Sensors, Actuators & Microcontrollers — Fully Covered!** 🚀